

The Network That Remembers With Its Wiring

A walk through the interactive explainer — synaptic plasticity as short-term memory, and where it lives in Pathway's BDH and BDH-CQ.

Open with a memory that is already running

Most explainers start with a blank canvas and a Run button. This one opens mid-thought: a 24×24 grid of synapses is already flickering as pairs of sparse patterns arrive, roughly one per second. Each arrival is one association — a key pattern and a value pattern — and it strengthens synapses through a single Hebbian write. On the right, the system continuously probes itself with a recent key and holds the retrieval next to the truth. Early on the two are nearly identical. Nothing is being animated; your browser is doing the arithmetic as you watch.

One sentence to falsify

The whole artifact serves a single falsifiable claim: a network can store recent associations in a fixed-size synaptic state through Hebbian writes — no gradient step, no per-token memory slot — but as associations accumulate, interference corrupts retrieval until recall fails. The design brief for this track asked for exactly this shape of sentence, one the learner can reproduce, test, or break. So section 2 hands you the controls to break it.

The write, up close

Present a pair and watch the matrix: every synapse between an active key unit and an active value unit gains strength. The equation is one line, $M \leftarrow \lambda M + vk^T$, and its consequence is one line too: after t writes, reading with cue q returns $Mq = \sum \lambda^{(t-i)}(k_i \cdot q) v_i$ — a similarity-weighted sum of every stored value. That innocent Σ is the whole story. At low load the matching term dwarfs the others; the section shows you the sum itemized, term by term, so you can watch the crosstalk terms grow as you load the memory.

The moment it breaks

Drag the load slider past ten or twelve pairs and the exact-match curve folds: same 24×24 state, same stored information (with $\lambda = 1$ nothing was overwritten or deleted), yet over half of probes now retrieve the wrong value. The panel decomposes one failing retrieval into its target term plus crosstalk. This is the capacity limit of additive associative memories, the one named by Schlag, Irie and Schmidhuber for linear transformers: endlessly adding associations to finite memory inevitably reaches a limit. Superposition is not storage.

Three lenses on one equation

The same live state renders as a neuron-synapse diagram (Hebb's rule, decay as forgetting), as fast weights (slow parameters frozen at inference, fast parameter-shaped memory rewritten by the input stream — the Schmidhuber-to-Titans lineage), and as linear attention (a matrix-state RNN: GLA, RWKV-7, DeltaNet, Mamba-2's SSD view). Three literatures, one update rule. The special case $\lambda = 1$ — purely additive accumulation — is the case the Pathway brief highlights for BDH-CQ: state that accumulates additively per demonstration.

Where BDH lives in this

For most topics a BDH section would be a stretch. Here it is the subject. The BDH paper states that inference-time working memory “entirely relies on synaptic plasticity with Hebbian learning using spiking neurons,” and reports monosemantic synapses and sparse, positive activations. BDH-CQ extends the family to demonstration-driven reasoning: inputs continuously update recurrent memory, no parameter updates at inference, no verbalized chain of thought; a 150M configuration reports 29.5% pass@2 on public ARC-AGI-1 at about \$0.0007 per task. The artifact's evidence table labels every one of these claims as developer-reported preprint results — and the toy itself is prominently labeled as an independent toy, not BDH.

Try it

Work top to bottom, predict before you drag, and finish with the 60-second test: the system stores one pair, proves perfect recall, stores forty, and hands you the same probe again. If you can explain why the second probe fails — interference, not deletion — you have taken the concept. The sandbox exposes every variable: state size, decay, sparsity, key overlap, cue noise, seed. The repository ships a NumPy reference implementation and a test proving the JavaScript in your browser matches it to one part in a billion.

Sources: arXiv 2509.26507 (BDH); 2608.09888 (BDH-CQ); 2102.11174 (linear transformers as fast-weight programmers); 2508.08435 (fast-weight programming primer, TMLR 2025); 2501.00663 (Titans); 2312.06635 (GLA); 2503.14456 (RWKV-7); 2407.04620 (TTT); 2405.21060 (Mamba-2/SSD).